

UnifAI Guardrails, Cluster & Adaptive Routing Guide

Security Rules, Providers, Mesh Cluster Sync & Dynamic Adaptive Routing (Thanglish + English)

1. System Architecture & Traffic Flow Map

This manual covers UnifAI's runtime security defense and dynamic performance steering. **Cluster Config** synchronizes rate-limit token buckets and circuit breaker trip signals across distributed nodes via port 7946 gossip. Every client request is screened by **Guardrail Rules** and executed against **Guardrail Providers** (Presidio, RE2 regexes, Bedrock) to prevent PII leakage and prompt injection attacks. Clean requests are dynamically steered by the **Adaptive Routing Dashboard** using a reinforcement learning Multi-Armed Bandit model to the fastest, lowest-latency upstream model, governed by **Adaptive Routing Settings**.

2. Detailed Feature Dissection (5 Core Features)

Guardrail Rules — Guardrail Rules (AI Safety Policies & Prompt Screening)

Route: </workspace/guardrails/configuration>

■ Non-Tech Perspective (Eliya Vilakkam):

- **Uruvagam (Analogy):** Airport baggage scanner & customs security checkpoint mathiri.
- **Vilakkam:** Customer allathu employee AI kitta thappana kelvi kettu, bad words, company secrets, credit card numbers, allathu jailbreak / prompt injection attacks seiyatha mathiri thadukkura security rules. Incoming prompts matrum outgoing responses-ah scan pannu, rules meera patta udane request-ah block pannum allathu PII data-vai asterisk (*) pottu mask pannum.
- **Business Value:** Zero data leakage (PII/Secrets), regulatory compliance (HIPAA/GDPR), and brand reputation protection from toxic AI responses.

■ Tech Perspective (Technical Architecture):

- **Backend Architecture:** Pre-request and post-completion content security policy evaluation engine in Go. Evaluates prompt scope (`all`, specific prompt IDs from Prompt Repository, or custom Common Expression Language (CEL) expressions). Dispatches content to linked Guardrail Providers in parallel. Rejects violations with HTTP 400 Bad Request / 403 Forbidden containing structured guardrail verdict codes (`PII\_DETECTED`, `PROMPT\_INJECTION`, `TOXICITY`).
- **Endpoints:** GET `/api/v1/guardrails/config`, PUT `/api/v1/guardrails/config`, POST `/api/v1/guardrails/test`

■■ Screen Layout & Interactive Elements (Thirai Koorugal):

1. Melpura Koorugal (Top Bar Controls):

- Header with ShieldAlert Icon: Visual indicator of content safety controls.
- Add Rule Button: Opens the rule configuration modal dialog.
- Rules Count Badge: Displays the number of active safety rules.

2. Mathiya Koorugal & Tables (Tabs & Views):

- Guardrail Rules Table: Columns for Rule Name, Active Toggle Switch, Prompt Scope (All, Selected Prompts, Custom CEL), Linked Providers summary, Evaluation Action, Actions Menu (Edit, Delete).
- Instant Toggle: Enable or disable individual rules without service downtime.

3. Keezhpura Koorugal & Sheets (Bottom Elements & Slide-Overs):

- GuardrailRuleDialog (Modal): Form configuring Rule Name, Description, Active Switch, Linked Guardrail Providers multiselect, Prompt Scope Selector (All Prompts, Pick Prompts, Custom CEL Expression), and CEL Editor textarea.
- Delete Rule Confirmation Modal: Prevents accidental removal of production safety guardrails.

■ Connections & Structure Map (Inaippugal):

- **Data-vai Perumidam:** Evaluates incoming requests in FastHTTP Proxy and checks against templates in Prompt Repository.
- **Iyakkum Koorugal:** Blocks violating requests, redacts sensitive text, triggers Alert Channels, and logs security incidents into LLM Logs and Audit Logs.
- **Production Use Case:** Preventing customer service chatbot users from extracting system prompts or submitting SQL injection payloads into AI text fields.

Guardrail Providers — Guardrail Providers (Security Engines & Regex Scanners)

Route: </workspace/guardrails/providers>

■ Non-Tech Perspective (Eliya Vilakkam):

- **Uruvagam (Analogy):** Security inspection machinery & scanner vendor brands mathiri (X-ray machine, metal detector brands).
- **Vilakkam:** Security rules run aagurathukku thevaiyana scanning engines (e.g. Presidio PII Engine, Llama-Guard Toxicity Engine, Lakera Prompt Injection Scanner, AWS Bedrock Guardrails, allathu Custom Regex Pattern matching engines) configure pandra edham. Namakku thevaiyana regex patterns (credit cards, Aadhaar, SSN, passwords) inga add pannikalam.
- **Business Value:** Customizable data defense; multi-layered scanning capability; adaptability to enterprise-specific regex patterns and third-party security vendors.

■ Tech Perspective (Technical Architecture):

- **Backend Architecture:** Multi-engine security scanner registry. Supports local regex compiled matching (Google RE2 engine), lightweight embeddings classifiers, and external gRPC/HTTP safety microservices (Microsoft Presidio, Meta Llama-Guard). Scanners execute concurrently using Go worker routines with strict execution timeout deadlines (< 15ms) to prevent latency spikes.
- **Endpoints:** GET `/api/v1/guardrails/config`, PUT `/api/v1/guardrails/config`, POST `/api/v1/guardrails/providers/{id}/test`

■ Screen Layout & Interactive Elements (Thirai Koorugal):

1. Melpura Koorugal (Top Bar Controls):

- Add Provider Button: Opens provider engine creation dialog.
- Navigation Tabs: Seamless switching between Rules Configuration and Providers Configuration.

2. Mathiya Koorugal & Tables (Tabs & Views):

- Guardrail Providers Table: Columns for Provider ID, Policy Name, Engine Type (Regex / Presidio / Bedrock), Pattern Count badge, Linked Rules count badge, Actions (Edit, Delete).
- Dependency Safety Check: Prevents deletion of any provider currently linked to an active rule.

3. Keezhpura Koorugal & Sheets (Bottom Elements & Slide-Overs):

- GuardrailProviderDialog: Form configuring Provider ID, Policy Name, Engine Type selector, and multi-line Regex Patterns textarea (one regex pattern per line, e.g. `\b\d{4}-\d{4}-\d{4}-\d{4}\b` for credit cards).`
- Pattern Syntax Validator: Real-time RE2 regex syntax checker with instant error highlighting.

■ Connections & Structure Map (Inaippugal):

- **Data-vai Perumidam:** Configured by DevSecOps engineers with enterprise regex patterns and API credentials.
- **Iyakkum Koorugal:** Supplies scanning algorithms and regex patterns to Guardrail Rules.

■ **Production Use Case:** Configuring custom regex patterns to detect and mask proprietary internal project code names (e.g. 'Project-Titanium', 'Apollo-Core') before prompts reach public OpenAI models.

Cluster Config — Cluster Config (Distributed Mesh & High-Availability Sync)

Route: </workspace/cluster>

■ Non-Tech Perspective (Eliya Vilakkam):

- **Uruvagam (Analogy):** Multi-aircraft flight formation allathu multi-branch bank network mathiri (ellam branches-um real-time sync-la irukkum).
- **Vilakkam:** Oru periya company-la lakshakanakana requests handle panna multiple UnifAI servers (USA, Europe, India regions) run aagum pothu, antha ellam servers-um onnukonnu pesikittu, rate limits, virtual key balances, and circuit breaker states instant-ah sync aaguratha uruthi seiyura master cluster control panel.
- **Business Value:** Zero single point of failure; multi-region active-active deployment; global consistency across worldwide AI infrastructure.

■ Tech Perspective (Technical Architecture):

- **Backend Architecture:** Distributed Mesh / Gossip protocol (HashiCorp Memberlist / Serf consensus). Cluster nodes communicate via TCP/UDP peer gossip over a dedicated clustering port (default: 7946). Synchronizes in-memory token rate-limit buckets, distributed circuit breaker states, and virtual key revocations with eventual consistency (< 50ms) across regions. Automatic split-brain avoidance and dead-node ejection.
- **Endpoints:** GET `/api/v1/cluster/config`, PUT `/api/v1/cluster/config`, GET `/api/v1/cluster/nodes`, POST `/api/v1/cluster/join`

■ Screen Layout & Interactive Elements (Thirai Koorugal):

1. Melpura Koorugal (Top Bar Controls):

- Header with Network Icon: Visual representation of distributed clustering.
- Cluster Mode Master Switch: Instant toggle between Standalone and Distributed Cluster mode.

2. Mathiya Koorugal & Tables (Tabs & Views):

- Cluster Topology Configuration Card:
- Cluster Type Selector: Mesh vs Gossip/Raft.
- Region Identifier Input: e.g. ``us-east-1``, ``eu-west-1``, ``ap-south-1``.
- Peer Seed List Textarea: Multi-line list of peer node addresses (``host:port``, e.g. ``10.0.0.12:7946``).
- Node Information Card: Node ID, Local Bind Address, Node Role (Leader, Follower, Peer), Active Healthy Peers count.

3. Keezhpura Koorugal & Sheets (Bottom Elements & Slide-Overs):

- Peer Syntax Validator: Real-time check verifying valid IP:Port formats with inline warning alerts.
- Save Configuration Button: Commits cluster topology with reload notification.

■ Connections & Structure Map (Inaippugal):

- **Data-vai Perumidam:** Configured by Cloud Infrastructure & Kubernetes DevOps teams.
- **Iyakkum Koorugal:** Synchronizes Virtual Key rate limits, Circuit Breaker trip states, and Adaptive Routing scores across all gateway instances.

■ **Production Use Case:** Deploying 10 load-balanced UnifAI gateway pods in Kubernetes, ensuring that a user's 60 RPM rate limit is globally enforced across all 10 pods rather than allowing 600 requests.

Adaptive Routing Dashboard — Adaptive Routing Dashboard (Live Health Scoring & Traffic Steering)

Route: </workspace/adaptive-routing>

■ Non-Tech Perspective (Eliya Vilakkam):

- **Uruvagam (Analogy):** Google Maps Live Traffic Navigation mathiri (Traffic irukkura road-ah thavirthu fast-ana road-la auto-reroute pandra mathiri).
- **Vilakkam:** Oru AI model (e.g. OpenAI GPT-4o) ippo slow aagiduchuna allathu error adikkithu na, system athai live-ah detect panni, automatically athe tharam konda innoru fast-ana provider-ku (e.g. Azure OpenAI allathu Anthropic Claude) traffic-ah thiruppi vittu user-ku semma fast response tharum. Ovvoru vendor-oda live health score-aiyum inga live-ah pakkalam.
- **Business Value:** Fastest AI response times for users; zero human intervention during vendor slowdowns; automated performance optimization.

■ Tech Perspective (Technical Architecture):

- **Backend Architecture:** Multi-Armed Bandit (MAB) reinforcement learning load balancer. Continuously evaluates provider directions and individual API key routes based on live telemetry (p50/p95 latency, error rates, throttle headers, HTTP status codes). Polled via UI every 8 seconds (`pollingInterval: 8000ms`). Dynamically adjusts traffic weighting coefficients in real time to steer traffic to the highest-scoring upstream paths.
- **Endpoints:** GET `/api/v1/load-balancer/routes`, PUT `/api/v1/load-balancer/config`, GET `/api/v1/load-balancer/metrics`

■ Screen Layout & Interactive Elements (Thirai Koorugal):

1. Melpura Koorugal (Top Bar Controls):

- Header with Gauge Icon: Adaptive Routing and Performance Steering.
- Settings Button: Deep-link to Adaptive Routing Settings (`/workspace/adaptive-routing/settings`).

2. Mathiya Koorugal & Tables (Tabs & Views):

- Summary Metric Cards (3 Cards):
 - Load Balancer Switch: Master toggle displaying Enabled/Disabled status with instant switch.
 - Scoring Strategy: Displays active algorithm (``latency_p95``, ``balanced``, ``least_errors``, ``cost_aware``).
 - Active Dynamic Routes Count: Total monitored provider directions.
- Live Adaptive Routes Table: Columns for Provider / Direction, Model Name, Health Score (0-100 color badge), p50 Latency (ms), p95 Latency (ms), Error Rate (%), Traffic Weight (%), Active Status badge.

3. Keezhpura Koorugal & Sheets (Bottom Elements & Slide-Overs):

- Auto-Refresh Poller: Live 8-second background polling reflecting shifting traffic weights as upstream latencies change.

■ Connections & Structure Map (Inaippugal):

- **Data-vai Perumidam:** Continuously ingests latency and error metrics from FastHTTP Proxy and Model Providers.
- **Iyakkum Koorugal:** Dynamically overrides Routing Rules traffic weights and cooperates with Circuit Breaker.

■ **Production Use Case:** During OpenAI latency spikes (jumping from 800ms to 4,500ms), adaptive routing automatically shifts 85% of traffic to Azure OpenAI instance maintaining a 650ms p95 latency.

Adaptive Routing Settings — Adaptive Routing Settings (Load Balancing Policy & Pruning Rules)

Route: /workspace/adaptive-routing/settings

■ Non-Tech Perspective (Eliya Vilakkam):

- **Uruvagam (Analogy):** Car cruise control & autopilot driving preference settings mathiri.
- **Vilakkam:** Adaptive load balancer eppadi nadanthukanum nu rules poduvom: Slow-ana vendor-ah automatic-ah ignore pannanuma? Oru vendor fail aana automatic-ah adutha vendor-ku reroute pannanuma? Multi-key balancing on pannanuma nu control pandra settings page.
- **Business Value:** Predictable routing behavior; fine-grained engineering control over automated failovers; custom tuning for sensitive production workloads.

■ Tech Perspective (Technical Architecture):

- **Backend Architecture:** Load balancer configuration and policy persistence engine. Manages `LoadBalancerConfig` stored in PostgreSQL. Controls direction selection algorithms, key-level round-robin weighting, failed direction rerouting policies, and automatic pruning of degraded fallback nodes from dynamic routing tables.
- **Endpoints:** GET /api/v1/load-balancer/config, PUT /api/v1/load-balancer/config

■■ Screen Layout & Interactive Elements (Thirai Koorugal):

1. Melpura Koorugal (Top Bar Controls):

- Header Title: Adaptive Routing Settings.
- Back Button: Return navigation to Adaptive Routing Dashboard (`/workspace/adaptive-routing`).
- Save Settings Button: Commits configuration updates.

2. Mathiya Koorugal & Tables (Tabs & Views):

- Settings Card with 5 Policy Toggle Rows:
 - 1. Enable adaptive load balancing: Master switch using live route scores when selecting provider and key.
 - 2. Direction selection: Automatically pick the healthiest provider for a requested model.
 - 3. Route selection: Weight individual API keys inside the chosen provider to balance quota.
 - 4. Reroute failed directions: If a pinned provider is unhealthy, automatically reroute the request to a healthy one.
 - 5. Prune failed fallbacks: Automatically drop unhealthy directions from a request's configured fallbacks list.

3. Keezhpura Koorugal & Sheets (Bottom Elements & Slide-Overs):

- Save Configuration Button (Save): Persists configuration with instant toast confirmation and state synchronization.

■ Connections & Structure Map (Inaippugal):

- **Data-vai Perumidam:** Configured by Lead Architects and DevOps engineers.
- **Iyakkum Koorugal:** Governs the live algorithmic behavior of Adaptive Routing Dashboard and upstream request dispatching.
- **Production Use Case:** Enabling 'Reroute failed directions' and 'Prune failed fallbacks' ensuring critical banking apps never experience timeouts due to stale upstream connections.

3. Cross-Feature Interconnections & Data Flow

Source Feature	Connected To	Data Flow & Trigger Action
Guardrail Rules	Guardrail Providers	Executes regex, Presidio, or Bedrock scanner engines linked to the rule.
Guardrail Rules	FastHTTP Proxy	Intercepts and scans incoming prompts and outgoing completions before forwarding.
Cluster Config	Virtual Keys	Synchronizes in-memory token rate-limit buckets across all cluster nodes.
Cluster Config	Circuit Breaker	Propagates tripped circuit breaker states globally within 50ms.
Adaptive Routing	Model Providers	Measures live latency and error rates across all vendor API keys.
Adaptive Routing	Routing Rules	Dynamically adjusts weights and bypasses slow routes based on live scores.
Adaptive Settings	Adaptive Routing	Defines the pruning, rerouting, and key weighting policies for the load balancer.

4. Tech vs Non-Tech Comparative Matrix

Feature	Non-Tech View (Manager / CFO Parvai)	Tech View (DevOps / Architect Parvai)
Guardrail Rules	Prompt injection, bad words, and company secrets AI kitta pogama thadukkuma?	CEL conditional content security engine scanning prompts/responses and returning 400/403 on violation.
Guardrail Providers	Custom regex patterns (credit cards, Aadhaar, internal codes) add panna mudiyuma?	Multi-engine scanner registry supporting RE2 regexes, Presidio, Llama-Guard, and Bedrock Guardrails.
Cluster Config	Multiple servers run aanaalum budget and rate limits correct-ah sync aaguma?	Distributed Mesh/Gossip consensus protocol (port 7946) synchronizing state across multi-region nodes.
Adaptive Dashboard	Oru AI vendor slow aana automatic-ah fastest backup vendor-ku reroute aaguma?	Real-time Multi-Armed Bandit load balancer calculating live health scores (0-100) from p95 latency.
Adaptive Settings	Slow vendor-ah thavirkka, fallbacks prune panna policies configure pannalama?	Persisted LoadBalancerConfig controlling direction selection, route selection, and failed route pruning.